chFnirs: Reference Manual

Function-by-function documentation

Dale J. Cohen

2026-04-16

Contents

1 Overview **3**
1.1 Function hierarchy . . . 3
1.2 Output column schema . . . 4
1.3 Common arguments . . . 4

2 Omnibus testing **6**
2.1 `run_omnibus_by_area` . . . 6
2.1.1 Usage . . . 6
2.1.2 Arguments . . . 6
2.1.3 Return value . . . 6
2.1.4 Details . . . 6
2.1.5 Example . . . 7
2.2 `run_omnibus_by_area_bh` . . . 8
2.2.1 Usage . . . 8
2.2.2 Arguments . . . 8
2.2.3 Return value . . . 8
2.2.4 Example . . . 8

3 Activation and slope tests **10**
3.1 `run_activation_tests` . . . 10
3.1.1 Usage . . . 10
3.1.2 Arguments . . . 10
3.1.3 Return value . . . 10
3.1.4 Example . . . 10
3.2 `run_slopes_vs_zero` . . . 11
3.2.1 Usage . . . 11
3.2.2 Arguments . . . 11
3.2.3 Return value . . . 11
3.2.4 Example . . . 11

4 Gated post-hoc contrasts **12**
4.1 `run_gated_posthoc` . . . 12
4.1.1 Usage . . . 12
4.1.2 Arguments . . . 12
4.1.3 Return value . . . 13
4.1.4 Example . . . 13
4.2 `run_gated_slopes` . . . 14
4.2.1 Usage . . . 14
4.2.2 Arguments . . . 14

4.2.3 Return value . . . 14
4.2.4 Decomposition rules . . . 15
4.2.5 Example . . . 15

5 Visualization **16**
5.1 `createHeatMapsFromLmerAreaAnalysis` . . . 16
5.1.1 Usage . . . 16
5.1.2 Arguments . . . 16
5.1.3 Return value . . . 17
5.2 `CreateSigBetaTable` . . . 18
5.2.1 Usage . . . 18
5.2.2 Arguments . . . 18
5.2.3 Return value . . . 18
5.3 `create_effect_size_heatmap` . . . 19
5.3.1 Usage . . . 19
5.3.2 Arguments . . . 19
5.3.3 Label modes . . . 20
5.3.4 Example . . . 20
5.4 `create_pvalue_heatmap` . . . 21
5.4.1 Usage . . . 21
5.4.2 Notes . . . 21
5.5 `create_omnibus_heatmap` . . . 22
5.5.1 Usage . . . 22
5.6 `create_posthoc_heatmap` . . . 22
5.6.1 Usage . . . 22
5.7 `create_forced_color_heatmap` . . . 23
5.7.1 Usage . . . 23
5.8 `plot_simple_slopes` . . . 24
5.8.1 Usage . . . 24
5.8.2 Key arguments . . . 24
5.8.3 Return value . . . 24
5.9 `plot_slope_lines` . . . 25
5.9.1 Usage . . . 25
5.9.2 Key arguments . . . 25
5.9.3 Notes . . . 25

6 Utilities **26**
6.1 `getBHvalues` . . . 26
6.1.1 Usage . . . 26
6.1.2 Arguments . . . 26
6.1.3 Return value . . . 26
6.1.4 Notes . . . 26
6.2 `getQvalues` . . . 26
6.2.1 Usage . . . 26
6.2.2 Return value . . . 26
6.3 `compute_cohens_d` . . . 27
6.3.1 Usage . . . 27
6.3.2 Arguments . . . 27
6.3.3 Return value . . . 27
6.4 `compute_partial_eta_sq` . . . 27
6.4.1 Usage . . . 27
6.4.2 Return value . . . 27
6.5 `has_tested_results` . . . 27
6.5.1 Usage . . . 27

6.5.2 Return value . . . 28
6.6 standardize_emmeans_columns . . . 29
6.6.1 Usage . . . 29
6.6.2 Arguments . . . 29
6.6.3 Common renames (all modes) . . . 29
6.7 write_results_by_type . . . 29
6.7.1 Usage . . . 29
6.8 sort_rois_anatomically . . . 29
6.8.1 Usage . . . 30
6.8.2 Return value . . . 30
6.9 shorten_string . . . 30
6.9.1 Usage . . . 30
6.10 run_lmer_analysis_by_area (deprecated) . . . 31

7 Internal functions **32**
7.1 iterate_by_area . . . 32
7.1.1 Usage . . . 32
7.1.2 Arguments . . . 32
7.1.3 Notes . . . 32

8 Acknowledgments **32**

1 Overview

This document provides function-by-function documentation for the `chFnirs` package. The package provides helper functions for second-level (group-level) analysis of functional near-infrared spectroscopy (fNIRS) data: unified wrappers for area-wise linear and mixed-model omnibus tests, `emmeans`-based activation tests and post-hoc contrasts, slope-based continuous-moderator analyses, Benjamini–Hochberg FDR correction, and p-value / effect-size / beta heatmap generation.

All analysis functions are designed to operate on long-format tables of per-subject, per-channel (or per-region) response estimates produced by any fNIRS first-level pipeline.

1.1 Function hierarchy

The package contains 26 exported functions (plus 1 internal helper) organized into five layers:

Layer	Functions	Role
Omnibus testing	`run_omnibus_by_area`, `run_omnibus_by_area_bh`	Area-wise Type III ANOVA with optional robust covariance and BH correction
Activation & slopes	`run_activation_tests`, `run_slopes_vs_zero`	Ungated marginal-mean and slope-vs-zero tests
Gated post-hocs	`run_gated_posthoc`, `run_gated_slopes`	Pairwise contrasts gated on omnibus significance

Layer	Functions	Role
Visualization	`createHeatMapsFromLmerAreaAnalysis`, `CreateSigBetaTable`, `create_pvalue_heatmap`, `create_posthoc_heatmap`, `create_omnibus_heatmap`, `create_forced_color_heatmap`, `create_effect_size_heatmap`, `plot_simple_slopes`, `plot_slope_lines`	Heatmaps and slope plots
Utilities	`getBHvalues`, `getQvalues`, `compute_cohens_d`, `compute_partial_eta_sq`, `has_tested_results`, `standardize_emmeans_columns`, `write_results_by_type`, `sort_rois_anatomically`, `shorten_string`, `run_lmer_analysis_by_area` (deprecated)	Effect sizes, p-value adjustment, column renaming, I/O
Internal	`iterate_by_area`	Shared iteration engine for Tier 2 analysis functions

1.2 Output column schema

All Tier 2 analysis functions produce output in a unified snake_case schema:

Identifier columns (all functions): `type`, `area`.

Omnibus: `effect`, `statistic`, `statistic_type` (`"F"`), `df_num`, `df_den`, `p_value`, `partial_eta_sq`, and (after BH) `p_bh`.

Activation / slopes-vs-zero: `predictor`, `estimate`, `se`, `df`, `statistic`, `statistic_type` (`"t"` or `"z"`), `p_value`, `cohens_d`.

Gated post-hocs: `contrast`, `predictor`, `estimate`, `se`, `df`, `statistic`, `statistic_type` (`"t"`), `p_bon`, `cohens_d`.

Gated slopes additionally: `omnibus_effect`, `posthoc_type`.

1.3 Common arguments

Several arguments are shared across the Tier 2 analysis functions:

Argument	Type	Description
`data`	data.frame	Long-format data with one row per subject x type x area x condition.
`formula`	formula or string	Model formula. Formula objects preferred; character strings silently coerced.
`area_col`	string	Column naming the brain area (e.g., `"ROI"`, `"Channel"`).
`type_col`	string	Column naming the measurement type (chromophore). Default: `"type"`.
`model`	string	Required, no default. `"lmer"` for linear mixed models, `"lm"` for ordinary linear models.

Argument	Type	Description
`vcov`	string or NULL	Covariance estimator. NULL resolves to `"classical"` for lmer, `"HC3"` for lm. Options: `"classical"`, `"HC0"`–`"HC5"`. HC with lmer raises an error.
`min_rows`	integer	Minimum cell size; smaller cells are silently skipped. Default: 5.

2 Omnibus testing

2.1 `run_omnibus_by_area`

Fits a linear or linear mixed model to every unique combination of measurement type and area, runs a Type III ANOVA on each fit, and returns a long-format table with one row per effect x type x area combination.

2.1.1 Usage

```
run_omnibus_by_area(
  data,
  formula,
  area_col,
  type_col = "type",
  model,
  vcov = NULL,
  min_rows = 5,
  effect_size = TRUE
)
```

2.1.2 Arguments

Argument	Type	Default	Description
`data`	data.frame		Long-format data.
`formula`	formula/string		Model formula.
`area_col`	string		Area column name.
`type_col`	string	`"type"`	Measurement-type column.
`model`	string	*required*	`"lmer"` or `"lm"`.
`vcov`	string/NULL	`NULL`	Covariance estimator. NULL resolves to `"classical"` (lmer) or `"HC3"` (lm).
`min_rows`	integer	`5`	Minimum observations per cell.
`effect_size`	logical	`TRUE`	Always includes `partial_eta_sq` column; `NA_real_` for lmer.

2.1.3 Return value

Data frame with columns: `type`, `area`, `effect`, `statistic`, `statistic_type` (`"F"`), `df_num`, `df_den`, `p_value`, `partial_eta_sq`. Returns `NULL` if no cell produced output.

2.1.4 Details

On the choice of HC estimator. HC0–HC5 are heteroscedasticity-consistent (sandwich) covariance estimators. HC3 divides each squared residual by $(1 - h_{ii})^2$ and is approximately a jackknife estimator. Long and Ervin (2000) showed HC3 has the best small-sample properties among HC0–HC3, which is why it is the resolved default when `model = "lm"`.

Why HC estimators do not apply to lmer models. HC estimators are defined against the OLS hat matrix and residual structure. Linear mixed models do not have a hat matrix in the same sense. Passing any

HC value with `model = "lmer"` raises an explicit error directing the user to either set `vcov = "classical"` or switch to `model = "lm"`.

2.1.5 Example

```
df.omnibus <- run_omnibus_by_area(
  data     = df.all,
  formula  = estimate ~ Group * cond + (1 | ID),
  area_col = "ROI",
  model    = "lmer"
)
```

2.2 run_omnibus_by_area_bh

Convenience wrapper that chains `run_omnibus_by_area()` with `getBHvalues()` so that the common "fit omnibus, then BH-correct" sequence is a single call. The result is identical to calling the two functions in sequence.

2.2.1 Usage

```
run_omnibus_by_area_bh(
  data,
  formula,
  area_col,
  type_col       = "type",
  model,
  vcov           = NULL,
  min_rows       = 5,
  effect_size    = TRUE,
  bh_group_col   = c("type", "effect"),
  bh_p_val_col   = "p_value"
)
```

2.2.2 Arguments

All arguments from `run_omnibus_by_area`, plus:

Argument	Type	Default	Description
`bh_group_col`	character vector	`c("type", "effect")`	Column(s) defining independent BH correction families.
`bh_p_val_col`	string	`"p_value"`	Raw p-value column to correct.

2.2.3 Return value

The omnibus table with an added `p_bh` column. Returns `NULL` if the underlying omnibus call produced no output.

2.2.4 Example

```
# Two-line "omnibus + BH + gated post-hoc" pattern:
df.omnibus <- run_omnibus_by_area_bh(
  data     = df.all,
  formula  = estimate ~ Group * cond + (1 | ID),
  area_col = "ROI",
  model    = "lmer"
)
df.ph.group <- run_gated_posthoc(
  omnibus_df      = df.omnibus,
  effect_name     = "Group",
  data            = df.all,
  formula         = estimate ~ Group * cond + (1 | ID),
```

```
  area_col        = "ROI",
  contrast_factor = "Group",
  model           = "lmer"
)
```

3 Activation and slope tests

3.1 `run_activation_tests`

For each type x area cell, fits the model and computes marginal means at each level of `by_factor` via `emmeans::emmeans()`, then tests each marginal mean against zero. The "activation from baseline" test for categorical designs.

3.1.1 Usage

```
run_activation_tests(
  data,
  formula,
  area_col,
  by_factor,
  type_col  = "type",
  model,
  vcov      = NULL,
  min_rows  = 5,
  cohens_d  = TRUE
)
```

3.1.2 Arguments

Argument	Type	Default	Description
`by_factor`	string		Factor whose marginal means are tested (e.g., `"Group"`, `"cond"`).
`cohens_d`	logical	`TRUE`	Always includes a `cohens_d` column via `compute_cohens_d()`.

All other arguments are the common set described in Section 1.3.

3.1.3 Return value

Data frame with columns: `type`, `area`, `predictor`, `estimate`, `se`, `df`, `statistic`, `statistic_type` (`"t"` or `"z"`), `p_value`, `cohens_d`. Returns `NULL` if no cell produced output.

3.1.4 Example

```
df.activ <- run_activation_tests(
  data      = df.all,
  formula   = estimate ~ Group * cond + (1 | ID),
  area_col  = "ROI",
  by_factor = "Group",
  model     = "lmer"
)
```

3.2 run_slopes_vs_zero

For each type x area cell, fits the model and uses `emmeans::emtrends()` to compute the slope of a continuous moderator (`trend_var`), then tests each slope against zero. The continuous-moderator analog of `run_activation_tests()`.

3.2.1 Usage

```
run_slopes_vs_zero(
  data,
  formula,
  area_col,
  trend_var,
  by_factor = NULL,
  type_col  = "type",
  model,
  vcov      = NULL,
  min_rows  = 5,
  cohens_d  = TRUE
)
```

3.2.2 Arguments

Argument	Type	Default	Description
`trend_var`	string		Continuous moderator whose slope is tested (e.g., `"cortisol"`). Must appear in `formula`.
`by_factor`	string/NULL	`NULL`	If NULL, tests a single overall slope per cell. If a factor name, tests a slope within each level.

3.2.3 Return value

Data frame with columns: `type`, `area`, `predictor`, `estimate`, `se`, `df`, `statistic`, `statistic_type` (`"t"`), `p_value`, `cohens_d`. The `predictor` column is labeled `"<trend_var>_Slope"` (overall) or `"<level>_<trend_var>_Slope"` (within `by_factor`).

3.2.4 Example

```
df.slopes <- run_slopes_vs_zero(
  data      = df.all,
  formula   = estimate ~ Group * cond * cortisol + (1 | ID),
  area_col  = "ROI",
  trend_var = "cortisol",
  by_factor = "Group",
  model     = "lmer"
)
```

4 Gated post-hoc contrasts

4.1 `run_gated_posthoc`

For each type x area cell, checks whether a named omnibus effect passed the significance gate, and if so, runs pairwise Bonferroni-adjusted contrasts via `emmeans::contrast()`. Non-significant cells optionally receive NA-filled placeholder rows for grey heatmap tiles.

4.1.1 Usage

```
run_gated_posthoc(
  omnibus_df,
  effect_name,
  data,
  formula,
  area_col,
  contrast_factor,
  by_var       = NULL,
  type_col     = "type",
  model,
  vcov         = NULL,
  p_col        = "p_bh",
  p_threshold  = 0.05,
  fill_empty   = TRUE,
  min_rows     = 5,
  cohens_d     = TRUE,
  adjust       = "bonferroni"
)
```

4.1.2 Arguments

Argument	Type	Default	Description
`omnibus_df`	data.frame		Output of `run_omnibus_by_area_bh()`. Must contain `p_col`.
`effect_name`	string		Omnibus effect to gate on (e.g., `"Group"`, `"Group:cond"`).
`contrast_factor`	string		Factor to contrast over (e.g., `"Group"`).
`by_var`	string/NULL	`NULL`	Nesting factor for conditional contrasts (e.g., `"cond"`).
`p_col`	string	`"p_bh"`	Column in `omnibus_df` to gate on. Errors if missing.
`p_threshold`	numeric	`0.05`	Gate threshold.
`fill_empty`	logical	`TRUE`	NA-fill non-significant cells for grey heatmap tiles.
`adjust`	string	`"bonferroni"`	Passed to `emmeans::contrast()`. Output column is always `p_bon`.

4.1.3 Return value

Data frame with columns: `type`, `area`, `contrast`, `predictor`, `estimate`, `se`, `df`, `statistic`, `statistic_type` (`"t"`), `p_bon`, `cohens_d`.

4.1.4 Example

```
df.posthoc <- run_gated_posthoc(
  omnibus_df      = df.omnibus,
  effect_name     = "Group:cond",
  data            = df.all,
  formula         = estimate ~ Group * cond + (1 | ID),
  area_col        = "ROI",
  contrast_factor = "Group",
  by_var          = "cond",
  model           = "lmer"
)
```

4.2 run_gated_slopes

For each type x area cell, checks gated significance of every omnibus effect involving a continuous moderator and, for those that pass, runs pairwise Bonferroni-adjusted slope comparisons via `emmeans::emtrends()` + `emmeans::contrast()`. Automatically decomposes interactions based on `categorical_factors`.

4.2.1 Usage

```
run_gated_slopes(
  omnibus_df,
  data,
  formula,
  area_col,
  trend_var,
  categorical_factors,
  effect_pattern = trend_var,
  type_col       = "type",
  model,
  vcov           = NULL,
  p_col          = "p_bh",
  p_threshold    = 0.05,
  fill_empty     = TRUE,
  min_rows       = 5,
  cohens_d       = TRUE,
  adjust         = "bonferroni",
  suffix         = "SlopeDiff"
)
```

4.2.2 Arguments

Argument	Type	Default	Description
`trend_var`	string		Continuous moderator whose slopes are compared.
`categorical_factors`	character vector		Categorical factors in the design (e.g., `c("Group", "cond")`). Used to decompose interactions.
`effect_pattern`	string	`trend_var`	Regex for selecting slope-containing effects from `omnibus_df`.
`suffix`	string	`"SlopeDiff"`	Appended to predictor labels to distinguish slope from mean comparisons.

4.2.3 Return value

Data frame with columns: `type`, `area`, `omnibus_effect`, `posthoc_type`, `contrast`, `predictor`, `estimate`, `se`, `df`, `statistic`, `statistic_type` (`"t"`), `p_bon`, `cohens_d`.

4.2.4 Decomposition rules

For each matching omnibus effect, the function inspects the effect name via `grepl()` against `categorical_factors`:

- **No categorical factors in effect** (e.g., just `trend_var`): single overall slope vs zero.
- **One categorical factor** (e.g., `Group:trend_var`): pairwise slope comparison across that factor.
- **Two categorical factors** (e.g., `Group:cond:trend_var`): decomposed into both conditional slicings (`~ Group | cond` and `~ cond | Group`), tagged with Title Case `posthoc_type` labels (e.g., `"GroupByCond"`, `"CondByGroup"`).

4.2.5 Example

```
# Filter omnibus to slope effects, then run gated slopes
df.omnibus.h <- df.omnibus.h.full[grepl(hVar, df.omnibus.h.full$effect), ]
df.h.posthoc <- run_gated_slopes(
  omnibus_df          = df.omnibus.h,
  data                = df.all,
  formula             = h_formula,
  area_col            = "ROI",
  trend_var           = hVar,
  categorical_factors = c("Group", "cond"),
  model               = "lmer"
)
```

5 Visualization

5.1 createHeatMapsFromLmerAreaAnalysis

High-level wrapper that generates heatmaps and CSV files from model results. Dispatches to `CreateSigBetaTable` for reshaping and then to the appropriate heatmap function based on `heatmap_type`. Supports per-type subdirectory output and anatomical ROI sorting.

5.1.1 Usage

```
createHeatMapsFromLmerAreaAnalysis(
  results_df, area_col, group_col, sigValue_col,
  dv_col          = "Beta",
  area_file_type = "Channel",
  output_prefix  = NULL,
  output_dir     = ".",
  split_by_type  = TRUE,
  type_col       = "type",
  significance_threshold = 0.05,
  heatmap_type   = "posthoc",
  scale_type     = "log",
  log_power      = 0.2,
  asterisk_sz    = 10,
  removeIntercept = TRUE
)
```

5.1.2 Arguments

Argument	Type	Default	Description
`results_df`	data.frame		Long-format model results.
`area_col`	string		Area column name.
`group_col`	string/NULL		Column to group by (e.g., `"type"`).
`sigValue_col`	string		Corrected p-value column.
`dv_col`	string	`"Beta"`	Column whose sign gives effect direction.
`area_file_type`	string	`"Channel"`	`"ROI"` for anatomical sorting.
`output_prefix`	string	`NULL`	Base name for output files.
`output_dir`	string	`"."`	Parent output directory.
`split_by_type`	logical	`TRUE`	Write per-type subdirectories.
`heatmap_type`	string	`"posthoc"`	`"omnibus"`, `"posthoc"`, or `"activation"`.
`scale_type`	string	`"log"`	`"log"` or `"linear"`.
`log_power`	numeric	`0.2`	Controls log scale intensity.
`asterisk_sz`	numeric	`10`	Asterisk size in mm.
`removeIntercept`	logical	`TRUE`	Drop intercept rows.

5.1.3 Return value

Nothing. Writes CSVs and PDFs under `output_dir`.

5.2 CreateSigBetaTable

Reshapes a long-format data frame of model results into a wide-format `predictor x area` table suitable for heatmap rendering. Supports three output modes: `"discrete"` (-1, 0, 1), `"continuous_signed"` (signed p-value), and `"continuous_unsigned"` (absolute p-value).

5.2.1 Usage

```
CreateSigBetaTable(
  df.data,
  areaColName     = "area",
  sigColName,
  dv_col          = "estimate",
  predColName     = NULL,
  group_col       = NULL,
  output_mode     = "discrete",
  sigThresh       = 0.05,
  removeIntercept = TRUE
)
```

5.2.2 Arguments

Argument	Type	Default	Description
`df.data`	data.frame		Long-format model results.
`areaColName`	string	`"area"`	Area column name.
`sigColName`	string	*required*	Significance column (e.g., `"p_bh"`, `"p_bon"`).
`dv_col`	string	`"estimate"`	Column whose sign gives effect direction.
`predColName`	string/NULL	`NULL`	Predictor column. Auto-detects `"predictor"` then `"Predictor"`.
`group_col`	string/NULL	`NULL`	Split column (e.g., `"type"`).
`output_mode`	string/logical	`"discrete"`	`"discrete"`, `"continuous_signed"`, or `"continuous_unsigned"`.
`sigThresh`	numeric	`0.05`	Threshold for discrete mode.
`removeIntercept`	logical	`TRUE`	Drop intercept rows.

5.2.3 Return value

Named list of wide data frames, one per level of `group_col` (or one named `"all_data"`).

5.3 create_effect_size_heatmap

Draws one heatmap of effect-size values (e.g., Cohen's d, partial eta-squared) per level of an optional grouping column. Uses a diverging blue–white–red palette when values include negatives (centered at zero), otherwise a sequential white–orange palette. Supports optional significance asterisks, numeric value labels, and anatomical ROI sorting.

5.3.1 Usage

```
create_effect_size_heatmap(
  results_df,
  area_col,
  es_col,
  p_col              = NULL,
  significance_threshold = 0.05,
  asterisk_size      = 6,
  show_values        = TRUE,
  area_file_type     = "Channel",
  group_col          = NULL,
  title              = NULL,
  low_color          = "blue",
  mid_color          = "white",
  high_color         = "red",
  seq_low_color      = "white",
  seq_high_color     = "darkorange",
  neg_label_digits   = 2,
  pos_label_digits   = 3,
  save_dir           = ".",
  file_prefix        = "EffectSize",
  fig_width          = 11,
  fig_height         = 8.5,
  landscape          = FALSE,
  split_by_type      = TRUE
)
```

5.3.2 Arguments

Argument	Type	Default	Description
`results_df`	data.frame		Long-format results with `predictor`, `area_col`, and `es_col`.
`area_col`	string		Area column (x-axis).
`es_col`	string		Effect-size column (e.g., `"cohens_d"`, `"partial_eta_sq"`).
`p_col`	string/NULL	`NULL`	P-value column for asterisks. NULL = no asterisks.
`significance_threshold`	numeric	`0.05`	P-value threshold for asterisks.
`asterisk_size`	numeric	`6`	Asterisk size in mm.
`show_values`	logical	`TRUE`	Print numeric effect sizes in cells.

Argument	Type	Default	Description
`area_file_type`	string	`"Channel"`	`"ROI"` for anatomical sorting via `sort_rois_anatomically()`.
`group_col`	string/NULL	`NULL`	Split column (one heatmap per group).
`split_by_type`	logical	`TRUE`	Route per-group PDFs to subdirectories.

5.3.3 Label modes

The four cell-label modes are controlled by `show_values` and `p_col`:

`show_values`	`p_col`	Result
`TRUE`	`NULL`	Values only (default)
`TRUE`	non-NULL	Values with asterisks on significant cells
`FALSE`	non-NULL	Asterisks only
`FALSE`	`NULL`	No text

5.3.4 Example

```
# Asterisks only, anatomically sorted ROIs
create_effect_size_heatmap(
  results_df  = df.activation,
  area_col    = "area",
  es_col      = "cohens_d",
  p_col       = "p_bh",
  show_values = FALSE,
  area_file_type = "ROI",
  group_col   = "type",
  file_prefix = "Activation_byCond"
)
```

5.4 create_pvalue_heatmap

Generates a continuous heatmap for signed p-values (range -1 to 1) with split red/blue legends for positive and negative effects. Supports linear or log-style scaling and significance asterisks.

5.4.1 Usage

```
create_pvalue_heatmap(
  table_data,
  data_col_indices,
  descriptor_col_indices,
  scale_type             = "linear",
  log_power              = 0.2,
  legend_breaks          = c(-0.05, -0.01, -0.001, 0.001, 0.01, 0.05),
  significance_threshold = 0.05,
  asterisk_size          = 6,
  pos_legend_title       = "Positive Effect",
  neg_legend_title       = "Negative Effect",
  low_color              = "blue",
  mid_color              = "white",
  high_color             = "red",
  save_dir               = ".",
  file_name              = "Adjusted_P_Value_Heatmap.pdf",
  fig_width              = 8.5,
  fig_height             = 11,
  title                  = "Adjusted P-Value Heatmap",
  landscape              = FALSE
)
```

5.4.2 Notes

- Input data is a wide-format table produced by `CreateSigBetaTable()` in `"continuous_signed"` mode.
- `data_col_indices` are numeric column indices for the area columns; `descriptor_col_indices` for the y-axis label column.
- Used by `createHeatMapsFromLmerAreaAnalysis()` for `heatmap_type = "activation"`.

5.5 create_omnibus_heatmap

Generates a single-color green gradient heatmap for unsigned p-values (e.g., from omnibus F-tests). Darker green = smaller p-value. Supports log scaling (default) and significance asterisks.

5.5.1 Usage

```
create_omnibus_heatmap(
  table_data,
  data_col_indices,
  descriptor_col_indices,
  scale_type             = "log",
  log_power              = 0.2,
  legend_breaks          = c(0.001, 0.01, 0.05, 0.10, 0.50),
  significance_threshold = 0.05,
  asterisk_size          = 6,
  low_color              = "darkgreen",
  high_color             = "white",
  save_dir               = ".",
  file_name              = "Omnibus_Heatmap.pdf",
  fig_width              = 8.5,
  fig_height             = 11,
  title                  = "Omnibus P-Value Heatmap",
  landscape              = FALSE
)
```

5.6 create_posthoc_heatmap

Extension of `create_pvalue_heatmap` that renders NA cells (untested ROIs where the omnibus gate was not passed) as grey tiles.

5.6.1 Usage

```
create_posthoc_heatmap(
  table_data,
  data_col_indices,
  descriptor_col_indices,
  scale_type             = "linear",
  log_power              = 0.2,
  legend_breaks          = c(-0.05, -0.01, -0.001, 0.001, 0.01, 0.05),
  significance_threshold = 0.05,
  asterisk_size          = 6,
  pos_legend_title       = "Positive Effect",
  neg_legend_title       = "Negative Effect",
  low_color              = "blue",
  mid_color              = "white",
  high_color             = "red",
  untested_color         = "grey80",
  save_dir               = ".",
  file_name              = "PostHoc_Heatmap.pdf",
```

```
  fig_width              = 8.5,
  fig_height             = 11,
  title                  = "Post-Hoc P-Value Heatmap",
  landscape              = FALSE
)
```

5.7 create_forced_color_heatmap

Renders a heatmap using a discrete color mapping (e.g., `-1` = blue, `0` = white, `1` = red). Used by `createHeatMapsFromLmerAreaAnalysis()` for the discrete effect tables.

5.7.1 Usage

```
create_forced_color_heatmap(
  table_data,
  data_col_indices,
  descriptor_col_indices,
  color_mapping = c("-1" = "blue", "0" = "white", "1" = "red"),
  save_dir      = ".",
  file_name     = "Forced_Color_Heatmap.pdf",
  fig_width     = 8.5,
  fig_height    = 11,
  title         = "Significance Direction Heatmap",
  landscape     = FALSE
)
```

5.8 plot_simple_slopes

For each significant type x area cell in a BH-corrected omnibus table, produces a pair of simple-slopes plots for a continuous x continuous interaction: one with `var1` on the x-axis and `var2` as the moderator (at Mean ± SD levels), and one with roles reversed.

5.8.1 Usage

```
plot_simple_slopes(
  omnibus_df, data, formula, area_col,
  var1, var2,
  effect_pattern   = NULL,
  type_col         = "type",
  model,
  vcov             = NULL,
  p_col            = "p_bh",
  p_threshold      = 0.05,
  gate             = TRUE,
  mod_sd_levels    = c(-1, 0, 1),
  y_label          = NULL,
  pdf_prefix       = NULL,
  output_dir       = ".",
  split_by_type    = TRUE,
  show_points      = FALSE,
  fig_width        = 7,
  fig_height       = 5
)
```

5.8.2 Key arguments

Argument	Type	Default	Description
`var1`, `var2`	string		The two continuous variables forming the interaction.
`effect_pattern`	string/NULL	`NULL`	Regex for matching omnibus effects. Auto-generates from var1/var2.
`gate`	logical	`TRUE`	Only plot significant cells.
`mod_sd_levels`	numeric vector	`c(-1, 0, 1)`	SD units for moderator levels.
`y_label`	string/NULL	`NULL`	Auto-detected from formula LHS.
`pdf_prefix`	string/NULL	`NULL`	If non-NULL, writes multi-page PDFs.
`show_points`	logical	`FALSE`	Overlay raw subject data. Default FALSE because moderator slices are synthetic.

5.8.3 Return value

Nested list of ggplot objects organized by type. Each element is `list(page_A = <ggplot>, page_B = <ggplot>)`.

5.9 plot_slope_lines

Companion plot for `run_slopes_vs_zero()` and `run_gated_slopes()`. For every significant row, draws the fitted slope line(s) with optional CI ribbons and raw-data overlay. Auto-detects activation vs. post-hoc mode from the `posthoc_type` column.

5.9.1 Usage

```
plot_slope_lines(
  results_df, data, formula, area_col, trend_var,
  type_col      = "type",
  model,
  vcov          = NULL,
  p_col         = "p_bh",
  p_threshold   = 0.05,
  by_factor     = NULL,
  suffix        = "SlopeDiff",
  show_ci       = TRUE,
  show_points   = TRUE,
  point_alpha   = 0.4,
  pdf_prefix    = NULL,
  output_dir    = ".",
  split_by_type = TRUE,
  fig_width     = 7,
  fig_height    = 5
)
```

5.9.2 Key arguments

Argument	Type	Default	Description
`trend_var`	string		Continuous moderator (x-axis of every plot).
`by_factor`	string/NULL	`NULL`	Level-column for activation mode.
`suffix`	string	`"SlopeDiff"`	Suffix to strip when parsing `predictor` labels.
`show_ci`	logical	`TRUE`	Draw pointwise 95% CI ribbon.
`show_points`	logical	`TRUE`	Overlay raw subject data, filtered per line.
`point_alpha`	numeric	`0.4`	Transparency of data points.

5.9.3 Notes

- **Activation mode** (no `posthoc_type` column): each significant row = one slope line.
- **Post-hoc mode** (`posthoc_type` present): each significant row = two slope lines (the pairwise contrast).
- Three-way decompositions parse the conditioning level from the `predictor` column.

6 Utilities

6.1 getBHvalues

Calculates Benjamini–Hochberg FDR-adjusted p-values. Supports grouping by one or more columns so that separate families of tests are controlled independently.

6.1.1 Usage

```
getBHvalues(df_data, group_col = NULL, p_val_col)
```

6.1.2 Arguments

Argument	Type	Default	Description
df_data	data.frame		Input data.
group_col	string/vector/NULL	NULL	Column(s) defining independent families.
p_val_col	string		P-value column name.

6.1.3 Return value

The input data frame with an added pBH column.

6.1.4 Notes

When multiple columns are supplied for group_col, uses interaction() internally to build a single grouping factor. Single-row groups copy the raw p-value as the BH-adjusted value.

6.2 getQvalues

Wrapper for qvalue::qvalue() to calculate Storey's q-values from p-values. Supports grouping.

6.2.1 Usage

```
getQvalues(df_data, group_col = NULL, p_val_col)
```

6.2.2 Return value

The input data frame with an added qVal column.

6.3 compute_cohens_d

Computes Cohen's d by dividing a vector of estimates by the residual standard deviation of the fitted model. The sign of d matches the sign of the estimate (since σ is always positive).

6.3.1 Usage

```
compute_cohens_d(model, estimate_col)
```

6.3.2 Arguments

Argument	Type	Description
`model`	fitted model	Must support `stats::sigma()`.
`estimate_col`	numeric vector	Marginal means, contrasts, or slopes on the response scale.

6.3.3 Return value

Numeric vector of Cohen's d values. `NA_real_` if σ is zero, NA, or undefined.

6.4 compute_partial_eta_sq

Computes partial eta-squared ($SS_{effect}/(SS_{effect} + SS_{residual})$) for every row of an ANOVA table. Auto-detects the sum-of-squares column name (`"Sum Sq"` or `"SS"`).

6.4.1 Usage

```
compute_partial_eta_sq(anova_table)
```

6.4.2 Return value

Numeric vector of partial η^2 values. `NA_real_` if no SS column or no Residuals row is found.

6.5 has_tested_results

Gate helper for conditional heatmap rendering. Returns `TRUE` when a results data frame has at least one non-NA p-value (`p_bon` checked first, then `p_value`).

6.5.1 Usage

```
has_tested_results(df)
```

6.5.2 Return value

Single logical value.

6.6 standardize_emmeans_columns

Renames columns of an `emmeans` output data frame to the canonical chFnirs snake_case schema.

6.6.1 Usage

```
standardize_emmeans_columns(df, mode, trend_var = NULL)
```

6.6.2 Arguments

Argument	Type	Description
`df`	data.frame	From `as.data.frame()` on an emmeans result.
`mode`	string	`"marginal"` (emmean → estimate), `"contrast"` (p.value → p_bon), or `"trend"` (trend_var.trend → estimate).
`trend_var`	string/NULL	Required when `mode = "trend"`.

6.6.3 Common renames (all modes)

`SE` → `se`, `t.ratio` / `z.ratio` → `statistic`. Factor-level columns and `df` pass through untouched.

6.7 write_results_by_type

Convenience wrapper around `write.csv()` that splits a long-format table by its `type` column and writes one CSV per type into its own subdirectory.

6.7.1 Usage

```
write_results_by_type(
  df,
  filename,
  output_dir    = ".",
  split_by_type = TRUE,
  type_col      = "type",
  row.names     = FALSE,
  ...
)
```

6.8 sort_rois_anatomically

Reorders a character vector of ROI names according to a neuroanatomical hierarchy: frontal → precentral → parietal → angular → temporal, then superior medial → superior → middle → inferior, then left before right.

6.8.1 Usage

```
sort_rois_anatomically(roi_vector)
```

6.8.2 Return value

Character vector of sorted ROI names. Unrecognized ROIs are placed at the end with a warning.

6.9 shorten_string

Replaces each word in a string with its first `num_letters` characters. Optionally removes specified substrings first.

6.9.1 Usage

```
shorten_string(input_str, num_letters = 2, stringsToRemove = NULL)
```

6.10 `run_lmer_analysis_by_area` (deprecated)

Deprecated

Original area-wise lmer omnibus function. Superseded by `run_omnibus_by_area()` which supports both lmer and lm models, robust covariance, and the unified output schema. Calls produce a deprecation warning via the `lifecycle` package pointing to the replacement.

7 Internal functions

7.1 `iterate_by_area`

Shared iteration engine used internally by all Tier 2 analysis functions. Loops over unique `type_col` x `area_col` combinations, subsets data, fits a model via `fit_fn`, extracts results via `extract_fn`, and row-binds the output. Errors for individual cells are caught with `tryCatch` and logged; the loop continues.

7.1.1 Usage

```
iterate_by_area(data, area_col, type_col, min_rows,
                fit_fn, extract_fn, fn_label)
```

7.1.2 Arguments

Argument	Type	Description
`fit_fn`	closure	`function(df_cell)` returning a fitted model.
`extract_fn`	closure	`function(fit, df_cell)` returning a per-cell data.frame.
`fn_label`	string	Used in error messages to identify the calling function.

7.1.3 Notes

This function is `@keywords internal` and is not exported. Users should call the public wrappers.

8 Acknowledgments

This document was drafted with assistance from Claude (Anthropic). All technical content reflects the authors' design decisions and domain expertise.